

While this may not seem like a huge improvement over having two simple global variables that could be used to do the same thing, the implications of this technique are enormous. You'll get a better sense of this in Chapter 4, where I'll discuss how to create and maintain a large number of instances made from a class.

Python Data Types Are Implemented as Classes

It might not surprise you that all built-in data types in Python are implemented as classes. Here is a simple example:

```
>>> myString = 'abcde'
>>> print(type(myString))
<class 'str'>
```

We assign a string value to a variable. When we call the `type()` function and print the results, we see that we have an instance of the `str` string class. The `str` class gives us a number of methods we can call with strings, including `myString.upper()`, `myString.lower()`, `myString.strip()`, and so on.

Lists work in a similar way:

```
>>> myList = [10, 20, 30, 40]
>>> print(type(myList))
<class 'list'>
```

All lists are instances of the `list` class, which has many methods including `myList.append()`, `myList.count()`, `myList.index()`, and so on.

When you write a class, you are defining a new data type. Your code provides the details by defining what data it maintains and what operations it can perform. After creating an instance of your class and assigning it to a variable, you can use the `type()` built-in function to determine the class used to create it, just like with a built-in data type. Here we instantiate a `LightSwitch` object and print out its data type:

```
>>> oLightSwitch = LightSwitch()
>>> print(type(oLightSwitch))
<class 'LightSwitch'>
```
